Personal Finance Management System

Project Documentation for Viva / Interview

Contents:

1. TECHNOLOGY STACK USED & DEFINITIONS

ASP.NET Web Forms (.NET Framework 4.8)

A Microsoft web framework where each page has an .aspx UI file and a .aspx.cs code-behind file. Events like button clicks run C# on the server. Used because it is simple for academic CRUD projects.

C#

Object-oriented language running on .NET. Used for all business logic: login, database operations, validation, reports.

SQL Server / SQL Server Express

Relational database management system. Stores users, income, expense, budgets, reminders, and notes in tables.

ADO.NET (SqlConnection, SqlCommand, SqlDataAdapter)

Data access layer in .NET. SqlConnection opens DB connection; SqlCommand runs SQL; SqlDataAdapter fills DataSet/DataTable for GridView.

Master Page (Index.Master)

Shared layout template containing navbar, CSS, and scripts. All pages inherit the same header/footer.

Session State

Server-side storage per user. Session['loginid'] stores logged-in user ID so every page knows who is logged in.

GridView Control

ASP.NET control to display tabular data from database. Supports edit, delete, and paging.

Bootstrap 5 + Bootswatch Darkly Theme

CSS framework for responsive layout, buttons, navbar, tables, and progress bars.

SweetAlert

JavaScript library for attractive popup alerts (success, error, warning) instead of browser alert boxes.

Chart.js

JavaScript chart library. Used on Home dashboard and P/L Report for bar and doughnut charts.

IIS Express

Lightweight web server used by Visual Studio to run the project locally during development.

Web.config

XML configuration file for connection strings, session timeout, and compilation settings.

PBKDF2 Password Hashing (Rfc2898DeriveBytes)

Cryptographic algorithm that converts plain passwords into secure hashes with salt. Passwords are never stored as plain text.

Parameterized SQL Queries

SQL with @parameters instead of string concatenation. Prevents SQL Injection attacks.

NuGet Package: Microsoft.CodeDom.Providers.DotNetCompilerPlatform

Allows dynamic compilation of C# in ASP.NET Web Forms project.

2. INTERVIEW / VIVA QUESTIONS & ANSWERS

Q: What is your project about?

A: Personal Finance Management is a web application that helps users track income and expenses, set monthly budgets, view dashboard summaries, generate profit/loss reports, and manage reminders and notes.

Q: Which technology stack did you use and why?

A: ASP.NET Web Forms with C# and SQL Server. Web Forms is suitable for form-based CRUD applications. SQL Server provides reliable relational storage. Bootstrap gives a modern UI quickly.

Q: Explain the architecture of your project.

A: Three-tier architecture: (1) Presentation - .aspx pages + Master Page, (2) Business Logic - .cs code-behind files and helper classes, (3) Data Layer - SQL Server database accessed via ADO.NET.

Q: What is a Master Page?

A: Index.Master defines common layout (navbar, CSS, scripts). Other pages use MasterPageFile='~/Index.Master' and put content inside ContentPlaceHolder. Avoids repeating navbar on every page.

Q: How does user authentication work?

A: On login, email is validated, password is verified using PasswordHelper (PBKDF2 hash). If valid, user ID is stored in Session['loginid']. Protected pages check Session and redirect to login.aspx if null.

Q: What is Session? Why use it?

A: Session stores user-specific data on the server between HTTP requests. HTTP is stateless; Session remembers who is logged in.

Q: How do you prevent SQL Injection?

A: All queries use SqlParameter (@email, @loginid, etc.) instead of concatenating user input into SQL strings.

Q: How are passwords secured?

A: PasswordHelper uses PBKDF2 with random salt and 10,000 iterations. Stored format: PBKDF2:iterations:salt:hash. Legacy plain-text passwords auto-upgrade on first login.

Q: What is ADO.NET?

A: Set of .NET classes for database access: SqlConnection (connect), SqlCommand (execute SQL), SqlDataReader (read rows), SqlDataAdapter (fill DataSet), ExecuteScalar (single value like SUM).

Q: What is GridView and how is Edit implemented?

A: GridView displays database rows. Edit uses OnRowEditing (set EditIndex), EditItemTemplate with TextBoxes, OnRowUpdating (save to DB), OnRowCancelingEdit (cancel). CommandField ShowEditButton adds Edit/Update/Cancel.

Q: Explain your database schema.

A: registration (users), income, expense, reminders, notes, budgets. All transaction tables have loginid foreign key linking data to the logged-in user so users only see their own records.

Q: What is the difference between Income Report, Expense Report, and P/L Report?

A: Income Report lists income transactions in date range. Expense Report lists expenses. P/L Report shows total income vs total expense, net savings, category breakdown, and charts.

Q: How does the Budget module work?

A: User sets monthly limit per expense category in budgets table. SQL JOIN with expense table calculates spent amount. Progress bar shows percentage used. Green/yellow/red indicates safe/warning/over budget.

Q: What is PostBack in Web Forms?

A: When user clicks a server button, form posts back to same page. Page_Load runs again; event handler (e.g. btnSave_Click) executes on server.

Q: What is !IsPostBack?

A: Checks if page is loading for first time (not a postback). Used to bind GridView only once on initial load, not on every button click.

Q: What validation is implemented?

A: Client-side JavaScript (SweetAlert) plus server-side ValidationHelper for email, phone, amount, date, and category before database insert.

Q: What is connection string?

A: In Web.config: Data Source (SQL server name), Initial Catalog (database name finance_mgt), Integrated Security=True (Windows auth).

Q: What improvements did you make?

A: Password hashing, server validation, dashboard with charts, budget module, P/L report, edit records, parameterized queries, README and SQL script.

Q: What is Chart.js used for?

A: Renders bar chart (income vs expense) and doughnut charts (category breakdown) on dashboard and P/L report using canvas elements.

Q: What are limitations of your project?

A: No mobile app, no bank API integration, single-user session only, Web Forms is older than ASP.NET Core MVC, no email notifications.

3. PROJECT WORKFLOW

3.1 User Registration Flow

1. User opens Register.aspx and fills name, email, contact, category, password.
2. JavaScript valid() checks fields on client side.
3. btnSave_Click runs server validation (ValidationHelper).
4. System checks if email already exists in registration table.
5. Password is hashed with PasswordHelper.HashPassword().
6. Record inserted into registration table.
7. User redirected to login.aspx with success message.

3.2 Login Flow

1. User enters email and password on login.aspx.
2. Server validates email format and non-empty password.
3. SELECT srno, password FROM registration WHERE email=@email.
4. PasswordHelper.VerifyPassword compares entered password with stored hash.
5. If old plain password, it is upgraded to hash (UpgradePasswordHash).
6. Session['loginid'] = user srno.
7. Redirect to Home.aspx.

3.3 Dashboard Flow (Home.aspx)

1. Page_Load checks Session['loginid']; if logged in, shows pnlDashboard.
2. LoadDashboard() calculates current month income and expense totals.
3. Net balance = income - expense.
4. Recent 5 transactions loaded via UNION of income and expense.
5. Budget progress loaded from budgets JOIN expense.
6. Chart.js scripts generated by ChartHelper for bar and doughnut charts.
7. Today's reminders shown from reminders table.

3.4 Income / Expense CRUD Flow

1. User must be logged in (else redirect login.aspx).
2. Add: form validated -> INSERT with loginid from Session.
3. List: SELECT * WHERE loginid=@loginid ORDER BY srno DESC -> GridView.
4. Edit: GridView EditIndex set -> user edits inline -> UPDATE with parameters.
5. Delete: DELETE WHERE srno=@srno AND loginid=@loginid (user can only delete own data).

3.5 Budget Flow

1. User selects month and expense category on budget.aspx.
2. Enters budget amount; server validates category and amount.
3. IF EXISTS UPDATE else INSERT into budgets table.
4. Grid shows budget vs spent using LEFT JOIN expense on category and date range.
5. Progress percentage and color (green/yellow/red) calculated in C#.

3.6 P/L Report Flow (Report.aspx)

1. User selects From and To dates.
2. Server validates date range.
3. SUM income and SUM expense for period.
4. Net savings = income - expense.
5. GROUP BY category for income and expense tables.

6. Chart.js renders summary bar chart and category doughnut charts.

3.7 Workflow Diagram (Text)

User Browser -> ASP.NET Web Forms Page (.aspx) -> Code-Behind (.aspx.cs) + Helpers (Session check, Validation) -> ADO.NET (SqlConnection, SqlCommand) -> SQL Server (finance_mgt database) -> Data returned to GridView, Labels, Charts

4. LINE-BY-LINE CODE MEANING & WHY USED

4.1 PasswordHelper.cs (Security)

Purpose: Hash and verify passwords securely using PBKDF2 algorithm.

Line 9-12: Constants - SaltSize 16 bytes, HashSize 32 bytes, 10000 iterations, Prefix 'PBKDF2:'

WHY: Standard secure settings; prefix identifies hashed passwords vs legacy plain text.

Line 14-25 HashPassword(): Generate random salt, derive hash, return formatted string.

WHY: Same password produces different hash each time due to unique salt.

Line 17-19 RNGCryptoServiceProvider.GetBytes(salt)

WHY: Cryptographically secure random salt prevents rainbow table attacks.

Line 22-24 Rfc2898DeriveBytes (PBKDF2)

WHY: Industry-standard slow hash function resistant to brute force.

Line 28-51 VerifyPassword(): Parse stored hash, re-derive key, compare.

WHY: Login verification without storing plain password.

Line 35-37 Legacy check: if not PBKDF2 prefix, compare plain text.

WHY: Backward compatibility for old users; upgraded on next login.

Line 72-84 SlowEquals(): Bitwise compare all bytes.

WHY: Prevents timing attacks that could leak password information.

4.2 login.aspx.cs (Authentication)

Line 10: SqlConnection from Web.config connectionStrings['connstr'].

WHY: Central DB configuration; reused across all pages.

Line 14-18: Check Session['RegisterSuccess'] after registration redirect.

WHY: Show success message on login page (Redirect clears previous page).

Line 23-33: Server-side email and password validation before DB call.

WHY: Client JS can be bypassed; server must validate always.

Line 36-37: SELECT with @email parameter (not string concat).

WHY: SQL Injection prevention.

Line 44: PasswordHelper.VerifyPassword entered vs stored.

WHY: Secure password check including hash and legacy support.

Line 49-51: NeedsRehash -> UpgradePasswordHash.

WHY: Migrate old plain passwords to PBKDF2 automatically.

Line 54: Session['loginid'] = userId.

WHY: Identifies logged-in user on all subsequent pages.

Line 55: Response.Redirect('Home.aspx').

WHY: Send user to dashboard after successful login.

Line 78: ClientScript RegisterStartupScript SweetAlert.

WHY: Show popup message without full page refresh for errors.

4.3 Home.aspx.cs (Dashboard)

Line 16-18: if Session['loginid'] != null -> pnlDashboard.Visible = true.

WHY: Dashboard only for authenticated users.

Line 19: if (!IsPostBack) - load data only on first page load.

WHY: Avoid rebinding on every postback unless needed.

Line 31-32: startOfMonth and startOfNextMonth date range.

WHY: SQL filter for current calendar month totals.

Line 35-37: SUM income, SUM expense, balance = income - expense.

WHY: Core financial summary for dashboard cards.

Line 42: lblBalance CssClass green if positive, red if negative.

WHY: Visual indicator of financial health.

Line 135-144: UNION ALL income + expense, TOP 5 ORDER BY date DESC.

WHY: Combined recent activity list from both transaction types.

Line 94-103: budgets LEFT JOIN expense for spent calculation.

WHY: Show budget limit vs actual spending per category.

Line 112-119: Calculate remaining and percent in C# loop.

WHY: Progress bar needs computed fields not in database.

Line 51-54: ChartHelper builds Chart.js script for bar and doughnut.

WHY: Visual analytics without server-side chart libraries.

4.4 budget.aspx.cs (Budget Module)

Line 17-20: Session check; redirect if not logged in.

WHY: Budget data is private per user.

Line 48-53: IF EXISTS UPDATE ELSE INSERT (upsert pattern).

WHY: One budget per category per month; update if user changes amount.

Line 54-57: Parameters @loginid, @month, @cat, @amt.

WHY: Safe SQL and ties budget to current user.

Line 88-97: JOIN budgets with expense on category and date range.

WHY: Calculate how much user has spent against each budget.

Line 106-113: Add remaining and percent columns dynamically.

WHY: GridView displays progress without extra DB columns.

Line 136-141 GetProgressClass: >=100 red, >=80 yellow, else green.

WHY: Bootstrap progress bar color reflects budget status.

4.5 Web.config (Configuration)

Line 7-8 connectionStrings name='connstr': SQL Server path and database finance_mgt.

WHY: Single place to change database server for entire application.

Line 11 targetFramework='4.8': Uses .NET Framework 4.8.

WHY: Matches project target in Visual Studio.

Line 13 sessionState timeout='30': Session expires after 30 minutes idle.

WHY: Security - auto logout inactive users.

4.6 Index.Master (Layout)

Master Page contains navbar links to all modules, Register/Login/Logout buttons, Bootstrap CSS (Bootswatch Darkly), SweetAlert JS, Bootstrap bundle JS, and StyleSheet1.css. ContentPlaceHolder1 is where each page injects its body content. WHY: DRY principle - one navbar for all pages; consistent look and feel.

4.7 income.aspx.cs - Save Pattern (Representative CRUD)

ValidationHelper.IsValidDate / IsCategorySelected / IsValidAmount run first. INSERT INTO income (in_date,

in_category, in_amount, in_remark, loginid) with @parameters. loginid from Session ensures user can only add to their account. BindGrid() refreshes GridView. SweetAlert confirms success. WHY: Same pattern used for expense, reminder, note with respective validations.

4.8 Report.aspx.cs - P/L Calculation

GetTotal() uses SUM(CAST(amount AS FLOAT)) with BETWEEN dates and loginid filter. GetCategoryBreakdown() uses GROUP BY category for pie/doughnut chart data. ChartHelper.BuildBarChart and BuildDoughnutChart output JavaScript for Chart.js. WHY: Separates calculation logic from UI; charts render client-side for performance.

Note: Full source code is in the Finance_management folder. This document explains the most important files for viva and interview preparation. Generated for Personal Finance Management System project.